

Module 3: Design, Cohesion & Coupling

Modularity Scales, SOLID Principles, GoF Design Patterns (Singleton, Factory, Observer)

Module III: Software Design Engineering, Modularity & Architecture

1. The Software Design Process (Architectural Design, Interface Design, Component Design)
2. Fundamental Design Principles (Abstraction, Information Hiding, Modularity, Refactoring)
3. Module Cohesion Taxonomy: Functional (Best) to Coincidental (Worst) Complete Scale
4. Module Coupling Taxonomy: Data (Best) to Content (Worst) Complete Scale
5. Object-Oriented Design Principles (SOLID: Single Responsibility, Open-Closed, Liskov, Interface, Dependency)
6. Unified Modeling Language (UML): Structural Diagrams (Class, Object, Component)
7. UML Behavioral & Interaction Diagrams: Sequence, State Machine & Activity Diagrams
8. Software Architectural Styles: Layered, Client-Server, Pipe-and-Filter, MVC, Microservices
9. GoF Design Patterns Taxonomy: Creational, Structural & Behavioral Paradigms
10. Creational Patterns: Singleton & Factory Method Implementations & UML
11. Structural & Behavioral Patterns: Adapter, Decorator, Observer & Strategy Patterns
12. Comprehensive Solved BIT Mesra & GATE Exam Question Bank (8 Questions)

Topic 3 & 4: Modularity (Cohesion vs. Coupling)

The primary objective of software modularization is to achieve **High Cohesion (Single Responsibility within a module)** and **Low Coupling (Minimal Interdependence between modules)**.

Cohesion Type (High to Low)	Definition & Internal Binding Characteristics	Quality Rating
1. Functional	All elements of a module contribute to executing a single, well-defined mathematical task (e.g., <code>computeSquareRoot()</code>).	★★★★★ (Ideal)
2. Sequential	The output data of one element serves as the direct input to the next element inside the module (e.g., <code>readRecordAndFormat()</code>).	★★★★☆ (High)
3. Communicational	Elements operate on the exact same input data structure or produce the same output structure (e.g., <code>updateAndPrintCustomerReport()</code>).	★★★☆☆ (Medium)
4. Procedural	Elements are grouped because they execute in a specific sequential algorithmic order (e.g., <code>readInputsThenValidate()</code>).	★★☆☆☆ (Fair)
5. Temporal	Elements are grouped simply because they are executed at the same time (e.g., system <code>startupInit()</code> or <code>shutdown()</code>).	★☆☆☆☆ (Poor)
6. Logical	Elements perform logically similar tasks selected by a control flag (e.g., a single function with a switch case for all math operations).	★☆☆☆☆ (Poor)
7. Coincidental	Elements have zero meaningful relationship; grouped purely by accident or arbitrary convenience.	✗ (Worst)

Coupling Type (Low to High)	Definition & Inter-Module Communication	Quality Rating
1. Data Coupling	Modules communicate strictly by passing elementary primitive data parameters (e.g., `int`, `float`).	★★★★★ (Ideal)
2. Stamp Coupling	Modules pass complex composite data structures (e.g., entire `CustomerRecord`), even when the callee only needs a few fields.	★★★★☆ (Good)
3. Control Coupling	One module passes a control flag (`bool`, `enum`) that dictates the internal execution flow of the other module.	★★★☆☆ (Poor)
4. Common Coupling	Multiple modules share and mutate global variables or shared global memory blocks.	★★☆☆☆ (Very Poor)
5. Content Coupling	One module directly accesses, modifies, or branches into the internal private code/memory of another module (e.g., modifying private memory).	✖ (Worst)

Topic 8: Software Architectural Styles

Architectural Style	Structural Organization & Communication	Primary Use Cases
1. Layered (N-Tier)	Organized hierarchically into horizontal layers (UI → Business Logic → Data Access → Database). Upper layers invoke lower layers.	Enterprise web applications, OS architectures.
2. Pipe-and-Filter	Data flows through a stream of discrete transformation filters connected by communication pipes.	Unix pipelines, compilers, video processing engines.
3. Model-View-Controller	Decouples business data (Model), user interface presentation (View), and input handling (Controller).	Interactive GUI applications (React, Angular, Django).
4. Microservices	Decomposes system into small, independently deployable services communicating via lightweight REST/gRPC APIs.	Scalable cloud-native enterprise systems (Netflix, Uber).

Top BIT Mesra Exam Questions & Answers (Module III)

Q1. Explain the SOLID design principles in Object-Oriented Software Engineering. (10 Marks)

- **S (Single Responsibility Principle):** A class should have only one reason to change, encapsulating a single responsibility.
- **O (Open/Closed Principle):** Software entities should be open for extension, but closed for modification (achieved via interfaces and polymorphism).
- **L (Liskov Substitution Principle):** Subtypes must be substitutable for their base types without altering program correctness.
- **I (Interface Segregation Principle):** Clients should not be forced to depend on interfaces they do not use (prefer small, client-specific interfaces over fat general ones).
- **D (Dependency Inversion Principle):** High-level modules should not depend on low-level modules; both should depend on abstractions.